

React PDF OCR + Multilingual AI Summary Project — Detailed R&D;

This document contains detailed research and development planning for building an AI-powered PDF analysis application using React. The platform will support PDF viewing, OCR-based text extraction, multilingual summarization, zoom controls, translation, and AI-powered document interaction.

Project Goals

Upload and render PDFs Extract text from digital and scanned PDFs Perform OCR on image-based documents Generate AI-powered summaries Support multiple languages Zoom in / zoom out PDF pages Allow page navigation and search Export summaries Optional AI Chat with PDF feature

Recommended Tech Stack

Frontend: React + Vite + Tailwind CSS + TypeScript

PDF Rendering: react-pdf, pdfjs-dist

OCR Engine: Tesseract.js

AI Layer: OpenAI API / Gemini API

Backend: Node.js + Express

Advanced: LangChain, OpenCV.js, Pinecone

PDF Rendering System

The project should use react-pdf and pdfjs-dist for rendering PDF files directly inside the browser. The rendering layer should support: Multi-page rendering Page navigation Zoom controls Responsive layout Page rotation Lazy loading

Zoom System

The zoom system should maintain smooth rendering while preserving readability. Recommended zoom limits: Minimum Scale: 0.5 Default Scale: 1.0 Maximum Scale: 3.0 Implementation idea: `const [scale, setScale] = useState(1); setScale(prev => prev + 0.2); setScale(prev => Math.max(prev - 0.2, 0.5));`

OCR Pipeline

The OCR pipeline should intelligently detect whether the PDF already contains selectable text.

Digital PDF: Extract embedded text directly Faster and more accurate **Scanned PDF:** Convert page to canvas/image
Run OCR using Tesseract.js Workflow: PDF Page → Canvas → OCR → Extracted Text

Multilingual OCR

Tesseract.js supports many languages: English (eng) Hindi (hin) Japanese (jpn) Chinese (chi_sim) Arabic (ara) Korean (kor) French (fra) Spanish (spa) Example: `Tesseract.recognize(image, 'eng+hin')`

AI Summarization System

The AI summarization system should: Read extracted text Chunk large documents Generate concise summaries
Translate summaries Extract keywords Create detailed reports Recommended models: GPT-4o GPT-4.1 Gemini

Backend Architecture

The backend should securely handle AI requests and API keys.

Recommended routes: POST /summarize POST /translate POST /chat The backend should: Validate requests Rate limit users Manage token usage Handle chunk processing

Frontend Architecture

Recommended structure: src/ **■■■■** components/ **■■■■** services/ **■■■■** hooks/ **■■■■** utils/ **■■■■** pages/ **■■■■** App.tsx
Important Components: PDF Viewer Toolbar OCR Panel Summary Panel Upload Component

Performance Optimization

OCR is computationally expensive. Recommended optimizations: Use Web Workers Lazy render PDF pages OCR only visible pages Cache OCR results Debounce search actions Chunk AI requests

Advanced Features

Potential advanced features: AI Chat with PDF Semantic Search Voice Summary Keyword Highlighting Export Summary as PDF/DOCX Document Comparison Research Paper Analyzer

Security Considerations

Never expose API keys on the frontend. Important practices: Store API keys in environment variables Use backend proxy requests Validate uploaded PDFs Apply upload size limits Prevent malicious file uploads

Deployment

Recommended deployment: Frontend: Vercel / Netlify Backend: Render / Railway / VPS Recommended production stack: React Node.js Express OpenAI API Tesseract.js

Resume Project Description

Developed a multilingual AI-powered PDF analysis platform using React, Tesseract.js, and OpenAI APIs capable of OCR extraction, intelligent summarization, translation, zoomable PDF viewing, and document interaction for scanned and digital PDFs.

Development Roadmap

Phase 1: React setup PDF rendering Zoom controls **Phase 2:** OCR integration Multi-language support **Phase 3:** AI summarization Translation Export system **Phase 4:** AI Chat with PDF Search system Optimization

Recommended Libraries Table

Purpose	Library / Tool
PDF Viewer	react-pdf
PDF Engine	pdfjs-dist
OCR	Tesseract.js
AI Summary	OpenAI API / Gemini
Styling	Tailwind CSS
Backend	Express.js
Image Processing	OpenCV.js
UI Components	shadcn/ui

Conclusion

This project is strong enough for: Internship portfolio Hackathons SaaS product development Freelancing Final year college projects The combination of OCR, multilingual AI summaries, and modern React architecture makes it highly impressive and industry-relevant.